

Maximum-SEO Blogger (Blogspot) Theme XML: A Complete Implementation Guide for a Thai-Language-Learning Educational Blog

TL;DR

- **Blogger already renders 100% of your content server-side as raw HTML before the first byte reaches a bot** — Googlebot, Bingbot, Yandexbot, and the AdSense crawler all receive complete text, tables, vocabulary, and FAQ answers without executing any JavaScript, because Blogger processes every `data:` tag, `b:if`, and `b:loop` on Google's servers and ships finished HTML. Your only real job is to (a) make sure the *static* educational text actually lives in `data:post.body` (not generated by quiz JS), and (b) stop wasting effort duplicating meta tags Blogger emits automatically.
- **The single highest-impact change is structural, not cosmetic:** wrap `data:post.body` in a semantic `<article>` with visible breadcrumb navigation, crawlable label/related-post `<a href>` links, and `<noscript>` fallback text around the quiz — then let Blogger's built-in `all-head-content` handle canonical/Open Graph/description, adding only the tags it does *not* emit (viewport, `<title>`, `og:type`, `twitter:card`, breadcrumb + LearningResource JSON-LD).
- **For AdSense approval**, the risk is "low-value/thin content," not the presence of a game. Surround the interactive quiz with 800–1,500 words of genuine instructional prose, comparison tables, and FAQ answers in raw HTML, add About/Contact/Privacy static pages, and mark the page up as `LearningResource / Course` rather than only `SoftwareApplication` so bots classify the site as education, not a tool.

Key Findings

1. **Blogger is server-side rendered by design.** Unlike a React/SPA site, Blogger's XML template is a server-side templating language. All `data:blog.*`, `data:post.*`, `b:if`, `b:loop`, and `b:includable` constructs are evaluated by Google's servers; the browser (and every crawler) receives fully-formed HTML. This means your goal of "rendered content from byte 1" is met automatically for everything in the template and post body — **provided the content is typed into the post HTML, not injected by your quiz JavaScript at runtime.**
2. **`<b:include data='blog' name='all-head-content' />` auto-emits more than most tutorials admit.** It automatically outputs: the Content-Type /charset meta, `generator=blogger`, favicon link, `<link rel='canonical'>` (using `data:view.url.canonical`), feed/Atom autodiscovery links, a conditional `<meta name='description'>` (when a Search Description exists), and — via a nested `openGraphMetaData` includable — `og:url`, `og:title`, `og:description`, and a `1200×630` `og:image`. It does **not** emit `<title>` or the viewport meta. Manually re-adding canonical/OG/description therefore creates **duplicate tags**, a documented real-world problem (the GitHub gist [pathawks/1343315](https://gist.github.com/pathawks/1343315) reports Blogger "automatically creates another og:title and og:description based on the contents of the page, and Facebook take[s] these because they are further down").
3. **Bing and Yandex do not reliably execute JavaScript; AI crawlers execute none.** Per Digital Position's JavaScript-crawlers analysis, "Googlebot can render JavaScript, but it does so in a second, delayed phase. Bing, DuckDuckGo, and AI crawlers like ChatGPT cannot process JavaScript and only see static HTML." Even for Googlebot, rendering happens in a delayed "second wave" — Google developer advocate Tom Greenaway stated at Google I/O 2018 that "Googlebot might index a page before rendering is complete and the final render can actually arrive several days later... then we perform another wave of indexing," and Onely's research pegs the median rendering delay at roughly 5 seconds. So any vocabulary, table, or FAQ answer that only appears after your quiz JS runs is effectively invisible to Bing/Yandex

and delayed/at-risk for Google. Static HTML is non-negotiable for the educational content.

4. **AdSense's bar is "original, useful, non-thin content with clear navigation and policy pages" — not a ban on interactive tools.** A quiz/game is fine *if* the page is content-rich around it. Per Geniee Group's AdSense guidance, the common rejection trigger is "Thin content: Articles with very little substance, typically below 300-400 words, that do not provide detailed or helpful information," and it recommends "Articles between 800 and 1,500 words."
5. **The Sitelinks Searchbox rich result was retired by Google on November 21, 2024.** Per the Google Search Central Blog ("Farewell, Sitelinks Search Box," Oct 21, 2024): "we'll be removing this visual element starting on November 21, 2024... This update will roll out globally across all search results, in all languages and countries," adding that "you can remove sitelinks search box structured data from your site... there's no need to do so." `WebSite + SearchAction` JSON-LD no longer produces a visible search box, though the markup is harmless and may still aid agent-based search. Don't prioritize it.
6. **Blogger has two template-variable namespaces.** The modern `data:view.*` (e.g., `data:view.isSingleItem`, `data:view.isHomepage`, `data:view.featuredImage`) and the legacy `data:blog.*` (e.g., `data:blog.pageType`, `data:blog.canonicalUrl`). Both work; modern v3 themes prefer `data:view.*`.

Details

1. Maximum HTML5 semantic structure Blogger supports

Blogger imposes **no restriction** on HTML5 semantic elements. You can freely use `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, and `<footer>` in the template body. The only hard requirements are: the document must be **well-formed XML** (every tag closed, attributes quoted), there must be at least one `<b:section>` containing a `<b:widget>`, and there must be a `<b:skin>` tag for CSS. [Hacking Dream](#) A recommended body skeleton:

```

<body>
  <header role='banner'>...site title, logo...</header>
  <nav role='navigation'>...visible menu of categories/labels...
</nav>
  <main>
    <b:section id='main' showaddelement='yes'>
      <b:widget id='Blog1' locked='true' title='Blog Posts'
type='Blog'>
        <!-- post body rendered here as <article> -->
      </b:widget>
    </b:section>
  </main>
  <aside role='complementary'>...related posts, labels...
</aside>
  <footer role='contentinfo'>...</footer>
</body>

```

Constraint preserved: keep your existing `b:widget id='...'` values and SEOBLOGSPOT section IDs exactly — Blogger validates widget IDs against the Layout configuration, and changing them breaks the theme.

2. Rendering the post body as a semantic `<article>`

Inside the `Blog1` widget, locate `<b:includable id='postBody' var='post'>` (or `post.body` usage) and wrap it:

```

<article class='post hentry' itemscope='itemscope'
itemtype='https://schema.org/LearningResource'>
  <header>
    <h1 class='post-title entry-title' itemprop='name'>
<data:post.title/></h1>
  </header>
  <div class='post-body entry-content'
itemprop='articleBody'>
    <data:post.body/>
  </div>
  <footer class='post-footer'>...labels, author...</footer>
</article>

```

Because `<data:post.body/>` is expanded server-side, all your vocabulary grids, comparison tables, embedded CSS, JSON-LD, and the

quiz markup arrive as raw HTML. **Do not change anything inside the post body itself** — your existing JSON-LD (SoftwareApplication, LearningResource, FAQPage, AggregateRating), quiz JS, and Thai Unicode all remain untouched. (Thai Unicode renders correctly as long as the template declares UTF-8, which `all-head-content` does.)

3. The `<head>` : what to add and what NOT to add

Critical reversal of common advice: because `all-head-content` already emits canonical + Open Graph + description, **do not duplicate them**. Two safe strategies:

- **Strategy A (recommended, lowest risk):** Keep `<b:include data='blog' name='all-head-content' />`. Add only what it omits.
- **Strategy B (full control):** Remove `all-head-content` and hand-code *everything* (charset, favicon, feeds, canonical, OG). Only do this if you need precise OG control; you take on responsibility for the tags Blogger was providing. (This is the documented fix when duplicate OG tags become a problem — the pathawks gist answer: "Just remove `<b:include data='blog' name='all-head-content' />` from your template.")

Recommended head for Strategy A:

```
<head>
  <meta charset='UTF-8' />
  <meta content='width=device-width, initial-scale=1'
name='viewport' />

  <!-- Title: blog name on index, page name elsewhere -->
  <b:if cond='data:view.isHomepage'>
    <title><data:blog.title/> – Learn Thai with
Quizzes</title>
  <b:else/>
    <title><data:view.title/> | <data:blog.title/></title>
  </b:if>

  <!-- Blogger emits charset, canonical, description,
og:url/title/description/image, favicon, feeds -->
  <b:include data='blog' name='all-head-content' />
```

```

<!-- Add ONLY tags Blogger does NOT emit -->
<b:if cond='data:view.isSingleItem'>
  <meta content='article' property='og:type' />
</b:if>
  <meta content='website' property='og:type' />
</b:if>
<meta content='en_US' property='og:locale' />
<meta expr:content='data:blog.title'
property='og:site_name' />

<!-- Twitter Cards (Blogger does not emit these) -->
<meta content='summary_large_image' name='twitter:card' />
<meta content='@yourhandle' name='twitter:site' />

<!-- Robots: index posts/homepage/pages; noindex thin
search/label/archive -->
<b:if cond='data:view.isHomepage or
data:view.isSingleItem'>
  <meta content='index, follow, max-snippet:-1, max-image-
preview:large, max-video-preview:-1' name='robots' />
</b:if>
  <meta content='noindex, follow' name='robots' />
</b:if>

<!-- hreflang x-default for an English-only blog -->
<link expr:href='data:view.url.canonical' hreflang='x-
default' rel='alternate' />
<link expr:href='data:view.url.canonical' hreflang='en'
rel='alternate' />

<!-- Resource hints: non-blocking -->
<link href='https://fonts.googleapis.com'
rel='preconnect' />
<link crossorigin='crossorigin'
href='https://fonts.gstatic.com' rel='preconnect' />
<link href='https://cdnjs.cloudflare.com' rel='dns-
prefetch' />

<b:skin><![CDATA[ /* critical CSS here */ ]]></b:skin>
</head>

```

Why noindex on label/archive/search pages: these are auto-generated listing pages that duplicate post snippets; indexing them dilutes crawl

budget and can trigger duplicate-content and thin-content signals.

`follow` keeps link equity flowing.

4. Visible, crawlable navigation (not just JSON-LD)

Google requires that breadcrumb JSON-LD **match a visible breadcrumb trail**, and that internal links be real `<a href>` anchors (JS click handlers are invisible to crawlers — Google's documentation notes it "discovers new URLs by following href attributes in anchor tags"). Add a visible breadcrumb on item pages:

```
<b:if cond='data:view.isSingleItem'>
  <nav aria-label='breadcrumb'>
    <ol>
      <li><a expr:href='data:blog.homepageUrl'>Home</a></li>
      <b:if cond='data:post.labels'>
        <b:loop values='data:post.labels' var='label'>
          <b:if cond='data:label.isLast'>
            <li><a expr:href='data:label.url'>
              <data:label.name/></a></li>
          </b:if>
        </b:loop>
      </b:if>
      <li aria-current='page'><data:post.title/></li>
    </ol>
  </nav>
</b:if>
```

Labels as visible links (the category/niche signal): loop

`data:post.labels` and output each as `<a expr:href='data:label.url'><data:label.name/></a>`. Use labels like "Thai Vocabulary," "Thai Grammar," "Beginner Thai" so bots infer the niche from real anchor text.

Related posts as static HTML: Blogger's default related-posts widgets are JavaScript/feed-driven and load *after* render — invisible to non-JS crawlers (Blogger's own feed widget docs confirm content "is dynamically loaded using Google AJAX API after blog is rendered in a browser"). For maximum SEO, link related posts manually inside each post body (you control that HTML), or use Blogger's label-based linking. At minimum,

ensure a visible, crawlable list of recent posts in the sidebar/footer via the standard `BlogList` / `PopularPosts` widgets (rendered server-side).

5. Template-level JSON-LD (server-side rendered)

Add these as `<script type='application/ld+json'>` blocks using `expr`: where dynamic. Because Blogger renders them server-side, bots see them immediately. **Keep your existing in-post JSON-LD unchanged** — these template-level blocks complement (don't replace) it.

Organization/Publisher (site-wide):

```
<script type='application/ld+json'>{
  "@context": "https://schema.org",
  "@type": "Organization",
  "name": "<data:blog.title/>",
  "url": "<data:blog.homepageUrl/>",
  "logo": "https://yourblog.com/logo.png"
}</script>
```

WebSite (+ optional SearchAction — note the Searchbox rich result was retired Nov 21, 2024, so this is low priority):

```
<b:if cond='data:view.isHomepage'>
<script type='application/ld+json'>{
  "@context": "https://schema.org",
  "@type": "WebSite",
  "name": "<data:blog.title/>",
  "url": "<data:blog.homepageUrl/>",
  "inLanguage": "en",
  "potentialAction": {
    "@type": "SearchAction",
    "target": "<data:blog.homepageUrl/>search?q=
{search_term_string}",
    "query-input": "required name=search_term_string"
  }
}</script>
</b:if>
```

BreadcrumbList (item pages, mirroring the visible trail): build via `b:loop` over `data:post.labels` with `position` integers starting at 1 (Home),

matching the visible breadcrumb exactly. Schema.org's convention is that "a breadcrumb list has an itemListOrder of `ItemListOrderAscending`... beginning with '1' for the first item."

Escaping note: inside JSON-LD you must escape `&` as `&` and avoid raw `</>`; quotes inside `expr:` strings use `"`. Test every block in Google's Rich Results Test.

6. Handling the quiz JS so AdSense sees education, not a "tool"

This is the make-or-break issue for your niche. Strategy:

- **Static instructional prose is the foundation.** Each post should carry 800–1,500 words of real teaching content (vocabulary lists with meanings/transliteration, grammar explanations, usage examples, comparison tables) as raw HTML *outside* the quiz container. The quiz becomes an *enhancement*, not the page's reason to exist.
- **`<noscript>` fallback around the game.** Google's lazy-loading guidance is to "provide fallback content... `noscript` tags," and Backlinko notes the pattern that "Yahoo uses a `<noscript>` tag... [to] show static product details for JavaScript-heavy pages." Apply the same to your quiz:

```
<noscript>
  <p>This interactive quiz tests Thai vocabulary for
  greetings. Sample:
  "สวัสดี (sà-wàt-dii) = hello". Enable JavaScript to play
  the interactive version.</p>
</noscript>
```

This guarantees the educational substance of the quiz exists even for non-JS crawlers and signals genuine learning content.

- **Schema for mixed content.** Mark the *page/post* as `LearningResource` (or `Course / Quiz` from schema.org) so the dominant signal is education. Keep your `SoftwareApplication` markup for the interactive widget, but don't let it be the only or top-level type — over-marking a page as software invites a "tool site" classification (structured-data best practice warns against "Over-markup: adding types that don't match the primary purpose of the

page"). `LearningResource` with `educationalLevel`, `learningResourceType: "quiz"`, and `teaches` properties tells bots precisely what the page does.

- **FAQPage answers in raw HTML.** Your FAQ answers must be visible text in the post body (they already are if typed into the editor), not generated by JS. This both feeds FAQ rich results and adds crawlable word count.

7. AdSense content-density and placement specifics

AdSense evaluates: originality, depth/word count, navigation clarity, presence of policy pages, and overall E-E-A-T (Experience, Expertise, Authoritativeness, Trustworthiness). For this blog:

- Publish a solid baseline of substantive posts before applying. Per Hike Web Solutions' 2026 guide, "20–30 high-quality long-form articles are usually sufficient," noting Google "does not reject based on article count — it evaluates content quality, originality, structure, trust signals."
- Add **About, Contact, and Privacy Policy** static pages — their absence is a frequent rejection cause.
- Keep ad density moderate. AdSense's historical guidance allows multiple ad units but warns against layouts where ads dominate content; for educational pages, place at most one ad after the intro, one mid-content, one after the conclusion.
- Use **responsive ad units** (`data-full-width-responsive='true'`) and load the AdSense script with `async` so it doesn't harm Core Web Vitals.
- Wrap ad slots in fixed-min-height containers to prevent layout shift (CLS) — a common "CLS mystery" is ad slots collapsing to 0px then expanding.
- Gate ads off non-content pages with `b:if` (e.g., don't show ads on error/search pages).

8. Core Web Vitals in Blogger XML

- **Critical CSS inline in `<b:skin>`**; defer non-critical CSS. Blogger's `<b:skin>` is inlined server-side, which is good for LCP.

- **Fonts:** use `font-display: swap` and `preconnect` to `fonts.gstatic.com` (shown above). Limit to 1–2 families/weights.
- **JS:** add `defer` or `async` to non-critical scripts; never let render-blocking JS sit in `<head>`. Your quiz JS should load with `defer` or at the end of the body. Wrap inline scripts in `// ... //]]></code> to avoid XML parse errors.
• Images: add <code>loading='lazy'</code> to below-the-fold images and explicit <code>width / height</code> (or <code>aspect-ratio</code>) to prevent CLS. Never lazy-load the LCP/hero image. Blogger image URLs support size parameters (e.g., <code>/s1600/</code>, <code>/w400/</code>) – serve appropriately scaled versions.
• INP: keep quiz event handlers lightweight; avoid long main-thread tasks.
• Targets (per Google Search Central's Core Web Vitals doc, updated Dec 10, 2025): "Largest Contentful Paint (LCP) under 2.5 seconds, Interaction to Next Paint (INP) under 200 milliseconds, and Cumulative Layout Shift (CLS) under 0.1," measured "at the 75th percentile of real user data."

</div>
<div data-bbox="276 494 650 510" data-label="Section-Header">
<h2>9. Page-type detection and 2025–2026 best practices</h2>
</div>
<div data-bbox="276 517 583 532" data-label="Text">
<p>Use the modern <code>data:view.*</code> booleans for clarity:</p>
</div>
<div data-bbox="276 544 749 786" data-label="Table">
<table>
<tr>
<th>Purpose</th><th>Modern condition</th><th>Legacy equivalent</th></tr>
<tr>
<td>Homepage</td><td><code>data:view.isHomepage</code></td><td><code>data:blog.url == data:blog.homepageUrl</code></td></tr>
<tr>
<td>Single post</td><td><code>data:view.isPost</code></td><td><code>data:blog.pageType == "item"</code></td></tr>
<tr>
<td>Static page</td><td><code>data:view.isPage</code></td><td><code>data:blog.pageType == "static_page"</code></td></tr>
<tr>
<td>Post or page</td><td><code>data:view.isSingleItem</code></td><td><code>data:blog.pageType in ["item", "static_page"]</code></td></tr>
<tr>
<td>Index/listing</td><td><code>data:view.isMultipleItems</code></td><td><code>data:blog.pageType == "index"</code></td></tr>
</table>
</div>`

Purpose	Modern condition	Legacy equivalent
Label page	<code>data:view.isLabelSearch</code>	<code>data:blog.searchLabel</code>
Archive	<code>data:view.isArchive</code>	<code>data:blog.pageType == "archive"</code>
Error 404	<code>data:view.isError</code>	<code>data:blog.pageType == "error_page"</code>

XML escaping rules (to avoid the dreaded "could not be parsed" error):

- `&` must always be written `&` (this is the #1 cause of Blogger XML errors — "The reference to entity 'version' must end with the ';' delimiter").
- Inside attribute values, `"` → `"`; `,` → `'` (Blogger auto-replaces these).
- Wrap all CSS in `<b:skin><![CDATA[ ... ]]></b:skin>` and all inline JS in `//<![CDATA[ ... ]//]>`.
- `>` and `<` should be `>` / `<` in text content (within CDATA they're literal; note CDATA cannot be nested).
- Use `expr:attr='...'` for any attribute whose value is a `data:` expression; concatenate strings with `+` and `"`.

Priority ranking by SEO impact

1. **(Highest) Static HTML content audit** — confirm all educational text exists without JS. Determines whether Bing/Yandex/AI crawlers see anything at all.
2. **Semantic `<article>` + visible breadcrumb + crawlable label links** — drives niche classification and internal-link discovery.
3. **Eliminate duplicate canonical/OG/description tags** — prevents Search Console warnings and conflicting signals.
4. **Per-page robots directives** (`noindex`, `follow` on listings) — protects crawl budget, kills thin-content/duplicate signals.
5. **`<noscript>` + LearningResource schema around quiz** — secures education classification for AdSense.

6. **Core Web Vitals** (critical CSS, deferred JS, lazy images) — a ranking factor and AdSense/UX signal.
7. **(Lowest) WebSite/SearchAction** — retired rich result; near-zero impact.

What Blogger CANNOT do (platform limits)

- **No server-side code** of your own (no PHP/Node) — but you don't need it; Blogger's own server renders the template.
- **No control over auto-generated label/archive/search URLs** or their meta titles beyond robots directives (HubSpot-style note applies: there's no native way to set unique meta on auto-generated listing pages).
- **Cannot remove the `?m=1` mobile parameter URLs** entirely (manage via canonical, which Blogger handles automatically).
- **Cannot fully customize the related-posts widget to be server-rendered** without manual in-post linking; the native widget is JS/feed-based.
- **Limited control over the exact OG image dimensions** (Blogger resizes the featured image to 1200×630).
- **Cannot edit robots.txt within the theme** — that's a separate Settings field.

What Blogger CAN do (and does well)

- Full server-side rendering of all template + post HTML → bots get complete content from byte 1.
- Automatic canonical, OG (`og:url / og:title / og:description / og:image`), description, charset, favicon, and feed links via `all-head-content` .
- Per-page-type conditional logic via `b:if / data:view.*` .
- Arbitrary HTML5 semantics, JSON-LD, and meta tags in the head/body.
- Server-rendered widget loops (`b:loop`) for post lists, labels, popular posts.

Recommendations

Stage 1 — Fix the foundation (highest SEO impact):

1. Wrap `data:post.body` in a semantic `<article` `itemtype="https://schema.org/LearningResource">`; add visible breadcrumb `<nav>` and crawlable label `<a href>` links.
2. Audit one published post with JavaScript disabled in Chrome DevTools (and via Search Console URL Inspection → "View Tested Page"). Confirm every vocabulary item, table, and FAQ answer is present **without** JS. If anything is missing, move it from quiz JS into static post HTML. (Also test via "View Source," not just DevTools Elements, which shows post-JS DOM.)
3. Remove any **duplicate** canonical/OG/description tags you previously hand-added — keep `all-head-content` and let it emit them; add only `<title>`, `viewport`, `og:type`, `og:locale`, `og:site_name`, and Twitter Card tags.

Stage 2 — Strengthen classification & AdSense readiness:

4. Add `<noscript>` learning-objective text around each quiz; ensure 800–1,500 words of static instructional prose per post.
5. Ensure About, Contact, Privacy Policy static pages exist and are linked in the footer.
6. Add template-level Organization, WebSite, and BreadcrumbList JSON-LD (keep in-post JSON-LD unchanged).
7. Set `noindex, follow` on label/archive/search pages; `index, follow` on posts/homepage/static pages.

Stage 3 — Performance & polish:

8. Inline critical CSS in `<b:skin>`; `defer` quiz JS; `loading='lazy'` + explicit dimensions on non-hero images; `font-display:swap` + `preconnect`.
9. Place responsive AdSense units (after intro, mid, after conclusion) in fixed-height containers; load script `async`; gate off non-content pages.
10. Validate everything: Rich Results Test (each schema type), PageSpeed Insights (CWV), Search Console URL Inspection (rendered vs. crawled), and Bing Webmaster Tools.

Benchmarks that would change the plan:

- If Search Console shows posts indexed but **content missing** in the rendered view → JS-dependent content remains; escalate Stage 1 step 2.
- If AdSense rejects for "low-value content" → increase word count/originality per post (toward the 800–1,500 word range) and reduce ad density before reapplying.
- If CWV field data fails LCP > 2.5s → the culprit is usually one oversized hero image or render-blocking font/JS; fix before adding features.
- If duplicate-meta warnings appear in Search Console → you still have hand-added tags conflicting with `all-head-content` ; remove the manual ones.

Caveats

- **The exact internal contents of `all-head-content` are not published by Google.** The tag-by-tag breakdown here comes from reverse-engineered community page-source captures (notably BloggerBasics101) and is consistent with real output, but Blogger could change it. **Always verify by viewing your own page source** ("View Source," not DevTools Elements) to see exactly which canonical/OG/description tags Blogger emits before deciding what to add or remove.
- **Sitelinks Searchbox** rich result was retired Nov 21, 2024 — the `SearchAction` markup is included for completeness/agent-search but will not render a visible search box.
- Several cited Blogger-tutorial sources are individual blogs of varying authority; the platform-mechanics claims (server-side rendering, escaping rules, `data:view.*` variables) are corroborated across multiple sources and Google's own JavaScript-SEO documentation, but specific code snippets should be tested in a **backup copy** of your theme before going live (Theme → Backup first).
- AdSense approval is discretionary and policy-driven; no template change *guarantees* approval — content quality and originality dominate.

- Blogger has two theme-engine versions (v2/v3); confirm which your theme uses (the `b:version` attribute on the `<html>` tag) since some `data:view.*` tags require the newer engine.